

Spring Boot 4

The first three Spring Boot documents built one service. This one covers what changes when there are several, and what that costs.

Contents

1. [Monolith or microservices](#)
2. [The shape of the system](#)
3. [Service discovery](#)
4. [The API gateway](#)
5. [Calling another service](#)
6. [Events instead of calls](#)
7. [One database per service](#)
8. [Observability across services](#)
9. [Testing](#)

1. Monolith or microservices

A microservice is a service that can be deployed on its own. Everything else follows from that one property, and anything that breaks it is not a microservice regardless of its size.

	Monolith	Microservices
Deployment	One unit	One per service
Database	Shared	One per service
Scaling	The whole application	The service that needs it
A failure	Takes everything down	Takes one service down
Debugging	A stack trace	A distributed trace
Consistency	One database transaction	A saga with compensation
Team size	Works to a point	Needs a team per service

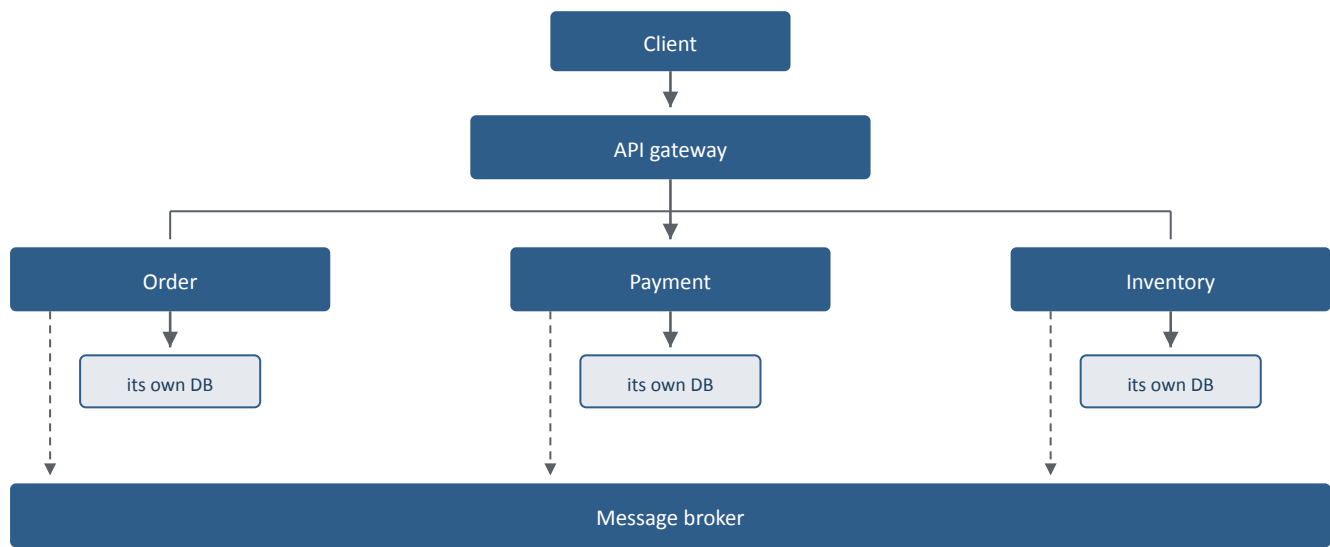
Split by business capability, not by layer. Order, payment and inventory are services. A "controller service" calling a "database service" is one application with a network in the middle.

The cost is real and it arrives first. Each service brings a pipeline, a database, a dashboard, an on-call rotation and a network hop between it and whatever it needs. Below a certain size that cost is larger than anything it buys. A modular monolith, with clear internal boundaries and one deployment, is the honest starting point for

most teams.

The failure to avoid has a name. A **distributed monolith** is a set of services that have to be released together, because they share a database or call each other in a chain. It carries every cost of both approaches and the benefit of neither.

2. The shape of the system



Solid lines are calls and dashed lines are events. No service reaches another service's database.

Two rules are visible in that picture, and both matter more than any framework choice. Nothing outside reaches a service directly, and no arrow crosses from one service into another's database.

3. Service discovery

Instances move, restart and change address. Nothing can hold a fixed host and port.

Approach	How it works
A registry, such as Eureka	Each service registers, and callers ask the registry for an address
The platform, such as Kubernetes	A service name resolves through DNS, and the platform balances the load

With Eureka, a service registers itself by name:

```
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://discovery:8761/eureka
```

and is then reachable by that name rather than an address:

```
restClient.get()
    .uri("http://payment-service/api/v1/payments/{id}", id)
    .retrieve();
```

On Kubernetes a registry is usually unnecessary. A Service object already gives a stable DNS name and balances across the healthy pods, so running Eureka as well means operating a second copy of something the platform does.

4. The API gateway

One address for everything outside, routing to the services behind it.

```
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service
          predicates:
            - Path=/api/v1/orders/**
        - id: payments
          uri: lb://payment-service
          predicates:
            - Path=/api/v1/payments/**
```

`lb://` means the address is resolved through discovery rather than written out.

Concern	Belongs in the gateway
Routing by path	Yes
TLS termination	Yes
Token validation	Usually, with services checking it again
Rate limiting	Yes
Request logging and trace start	Yes
Business rules	No

Business logic in the gateway undoes the point of the split. It becomes a component every team has to change and every release has to coordinate, which produces exactly the distributed monolith described above.

5. Calling another service

Client	Use it
<code>RestTemplate</code>	Existing code only. It is in maintenance
<code>RestClient</code>	The default for a blocking call, since Spring 6.1
<code>WebClient</code>	When the call must not block a thread
HTTP interface	A declarative interface over either of the two above

```
@Service
public class PaymentGateway {

    private final RestClient restClient;

    public PaymentGateway(RestClient.Builder builder) {
        this.restClient = builder.baseUrl("http://payment-service").build();
    }

    public PaymentResult charge(ChargeRequest request) {
        return restClient.post()
            .uri("/api/v1/charges")
            .body(request)
            .retrieve()
            .body(PaymentResult.class);
    }
}
```

The declarative form describes the call and lets Spring implement it:

```
public interface PaymentClient {

    @PostExchange("/api/v1/charges")
    PaymentResult charge(@RequestBody ChargeRequest request);
}
```

Every remote call needs a timeout, and the default is often none. Without one, a dependency that stops answering holds the calling thread until something else gives up, which is how one slow service takes down three.

```

RestClient.builder()
    .requestFactory(ClientHttpRequestFactoryBuilder.detect()
        .withConnectTimeout(Duration.ofSeconds(2))
        .withReadTimeout(Duration.ofSeconds(5))
        .build())
    .build();

```

A timeout, a limited retry and a circuit breaker belong together on every call that leaves the process.

6. Events instead of calls

A synchronous call couples the caller to the callee being up right now. An event does not.

```

@Service
public class OrderService {

    @Transactional
    public void place(Order order) {
        repository.save(order);
        events.send("orders", new OrderPlaced(order.getId(), order.getTotal()));
    }
}

```

```

@Component
public class OrderPlacedListener {

    @KafkaListener(topics = "orders", groupId = "inventory")
    public void on(OrderPlaced event) {
        inventoryService.reserve(event.orderId());
    }
}

```

Synchronous call	Event
The caller waits for an answer	The caller continues
Both sides must be up	Only the sender must be up
A failure travels back at once	The consumer retries later
One trace, easy to follow	Needs a correlation id to follow

That method has a bug, and it is the one to know. It writes to the database and sends to the broker, and nothing makes those two atomic. If the send fails after the commit, the order exists and nobody hears about it. If the commit fails after the send, the event describes an order that does not exist.

The fix is the **outbox**: write the event into a table in the same transaction as the order, and let a separate process read that table and publish. One transaction, one source of truth, and the publisher can retry safely.

7. One database per service

A service owns its data, and no other service reads its tables.

This is the rule that makes the rest work. A shared database means a schema change has to be agreed with every team that reads it, and released with them, which removes the independent deployment that defined a microservice in the first place.

Two things follow immediately, and both are uncomfortable at first.

There are no joins across services. A screen showing an order with its customer name calls two services and joins in the caller, or keeps a copy of the field it needs.

There are no transactions across services either. A business operation that spans three services becomes a **saga**: a sequence of local transactions, each with a compensating action.

```
place order → charge payment → reserve stock → confirm order
                                   ↓ fails
                        refund payment ← cancel order
```

Style	How the steps are driven
Choreography	Each service listens for an event and reacts. No coordinator
Orchestration	One service holds the sequence and calls each step

Choreography suits three or four steps. Beyond that, nobody can say what the sequence is without reading every service, and an orchestrator is clearer.

Compensation is not a rollback. A refund is a new transaction that a customer can see on a statement, not the erasing of the original charge. The business has to accept that intermediate state exists.

8. Observability across services

One user action now touches four services, so a stack trace from one of them explains very little.

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
```

With that on the classpath every log line carries the application name, a trace id and a span id:

```
2026-09-15T10:22:31.442Z INFO [order-service,5f3a9c2b1d4e,8e7f2a1c]
c.e.order.OrderService : placing order 41
```

The trace id is created at the first service to see the request and travels with it in the `traceparent` header. Spring propagates it through `RestClient`, `WebClient` and Kafka without any code.

Signal	Answers
Logs	What happened in one service
Metrics	How often, and how slow, across all of them
Traces	Where this one request spent its time

One habit makes the difference: put the trace id in the response. A support report then arrives with the identifier already attached, and finding the request becomes one search rather than a guess about timestamps.

9. Testing

Level	What it proves	Cost
Unit test	One class behaves	Milliseconds
<code>@SpringBootTest</code> with Testcontainers	One service works against its real database	Seconds
Contract test	Two services still agree on the interface	Seconds
End to end	The whole system works	Minutes, and it fails for unrelated reasons

Contract testing is the level that only exists here. The provider publishes what it promises, the consumer builds against a stub of that promise, and the provider's own build fails if it stops keeping it.

```
Contract.make {
  request {
    method POST()
    url '/api/v1/charges'
    body([ orderId: 41, amount: 250.00 ])
  }
  response {
    status 200
    body([ status: 'CHARGED' ])
  }
}
```

That replaces most end-to-end tests, and it runs in the provider's pipeline rather than in a shared environment that is often broken.

The pieces together

Piece	What it answers
Service boundary	What can be deployed on its own
API gateway	One address for everything outside
Service discovery	Where is the other service right now
<code>RestClient</code> with timeouts	How this service calls that one
Events and the outbox	How a fact reaches other services reliably
One database per service	Why the teams can release separately
Saga and compensation	What replaces a transaction across services
Trace id	How one request is followed through four services
Contract tests	Whether two services still agree

For example, imagine a food delivery system. Order, payment and delivery are three services, each with its own database and pipeline. The mobile app reaches one gateway, which validates the token and routes by path. Placing an order writes the order and an outbox row in one transaction, and a publisher turns that row into an `OrderPlaced` event. Payment and delivery both consume it. A failed charge emits `PaymentFailed`, and the order service compensates by canceling the order rather than undoing anything. Every log line from all three carries the same trace id.

The most useful thing at this point is not to memorize the patterns. Take one synchronous call between two services and write down what happens to the caller when the callee takes thirty seconds to answer.